



Ing. Fernando Negozi

fernando.negozi@eletica.it

www.eletica.it





PROCESSI

Linux ha la possibilità di gestire sia i processi che i job

Processo:

- Sequenza di istruzioni contenute in un eseguibile (In pratica un processo è un programma)
- Può avviare altri processi (detti figli) attraverso una syscall **fork**
- Il processo padre è responsabile dei processi figli (allocazione risorse, terminazione, messaggio fine esecuzione)
- Il sistema operativo permette l'esecuzione parallela dei processi
- Ogni processo, eccetto init, è generato da un padre.
- Ad ogni processo è associato un codice identificativo

Modalità di esecuzione

User mode : Modalità di esecuzione dei programmi utente

- Sono adottate politiche di sicurezza per evitare l'accesso a zone critiche del sistema ed ha a disposizione solo un sottoinsieme delle istruzioni e delle risorse hardware disponibili.
- Le richieste per utilizzare le risorse protette (come l'accesso diretto alla memoria fisica o alle periferiche hardware) vengono intercettate dal sistema operativo e gestite in modo sicuro.

Kernel mode Modalità per eseguire le operazioni del OS

- Modalità di esecuzione privilegiata che permette di accedere a tutte le risorse del sistema del kernel
- Modalità di esecuzione delle chiamate di sistema (System Calls)

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.0	2324	1500	?	Sl	22:37	0:00	/init
root	4	0.0	0.0	2340	68	?	Sl	22:37	0:00	plan9 --control-
root	8	0.0	0.0	2328	108	?	Ss	22:37	0:00	/init
root	9	0.0	0.0	2344	112	?	R	22:37	0:00	/init
pis	10	0.0	0.0	6212	5044	pts/0	Ss	22:37	0:00	-bash
pis	133	0.0	0.0	7480	3084	pts/0	R+	22:57	0:00	ps aux

USER: il nome utente che ha avviato il processo

PID: l'ID del processo

%CPU: la percentuale di CPU utilizzata dal processo

%MEM: la percentuale di memoria utilizzata dal processo

VSZ: (Virtual Set Size) la dimensione virtuale del processo (in kilobytes)

RSS: (Resident Set Size) la dimensione residente del processo (in kilobytes)

TTY: la console associata al processo

STAT: lo stato del processo (es. S: in esecuzione, Z: zombie)

START: l'ora di avvio del processo

TIME: il tempo di CPU impiegato dal processo fin a quel momento

COMMAND: il comando o il nome del processo in esecuzione

Init

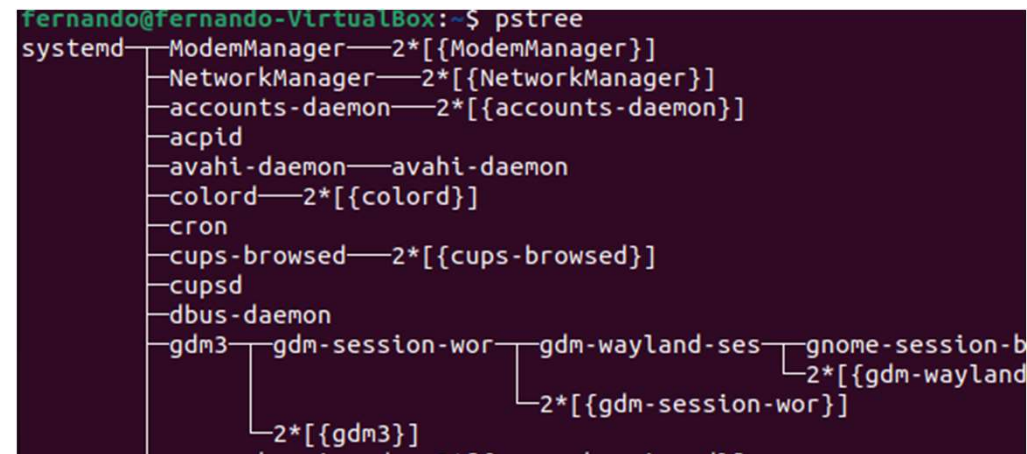
USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.0	2324	1500	?	Sl	22:37	0:00	/init

Processo principale (root)

- PID =1 - Primo processo che viene eseguito durante l'avvio del sistema operativo. Inizializza il sistema
- Avvia i servizi e tutti gli altri processi che permettono utilizzo del sistema (processi dei servizi di rete, servizi di login, servizi di GUI, etc.)
- Sempre attivo
- Spegnerne questo processo equivale a spegnere il sistema



Gerarchia dei processi



creazione

fork(): consente a un processo di generare un processo figlio (crea in memoria una copia del padre)

exec(): crea nuovi processi le cui istruzioni e variabili siano diverse da quelle del processo p

getpid(): consente di ottenere il PID

getppid(): consente di ottenere il PID del processo padre

Exit Termina in maniera incondizionata uno script.

Il comando **exit** opzionalmente può avere come argomento un intero che viene restituito alla shell come exit status dello script

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.0	2324	1500	?	Sl	22:37	0:00	/init

RSS : resident set size

quantità di memoria fisica (RAM) che un processo utilizza attualmente e che non viene condivisa con altre applicazioni.

VSZ : virtual set size

quantità complessiva di memoria virtuale (RAM e spazio su disco) che un processo potrebbe teoricamente utilizzare

Esempio

se un processo sta utilizzando un server Web, gran parte dello spazio di memoria virtuale utilizzato potrebbe essere costituito dal codice e dalle librerie del software del server anziché dall'applicazione specifica e da altri dati.

In questo caso, il VSS sarà maggiore del RSS.

Stato dei processi

STAT	STATO	Descrizione
R	in esecuzione (running)	Il processo è in esecuzione o è pronto ad essere eseguito cioè in attesa che gli venga assegnata la CPU
S	in pausa o dormiente (interruptable sleeping)	In attesa di un evento o di una risposta dal sistema. Può essere interrotto da un segnale
D	in pausa o dormiente (uninterruptable sleeping)	In attesa di un evento o di una risposta dal sistema. Non può essere interrotto da un segnale
W	In attesa (Waiting)	Rimosso dalla coda dei processi. Non utilizza memoria -Spostato nella memoria virtuale.
T	sospeso (Traced)	Il processo è stato fermato. Ad esempio da un debugger
Z	defunto (zombie)	È terminato ma il suo stato non è ancora stato letto dal padre. O il genitore è morto e non può essere cancellato. (provvederà Init)
N	bloccato (nice positivo)	Processo rallentato o interrotto in modo inaspettata (bug o conflitti) - consumo di risorse (nice positivo).
X	morto (dead)	Stato non valido

Ad ogni processo è attribuita una priorità che ne determina la schedulazione (tempo assegnato)

Processi **REAL TIME**: Priorità statica

processi **CONVENZIONALI**:

Priorità statica nulla – Priorità dinamica con RANGE -20 : +19 (valore di niceness)

Il valore di nice è un attributo numerico di ciascun processo dei sistemi linux

NICE

permette di cambiare la priorità di esecuzione di un processo.

sintassi: *nice* [OPTION] [COMMAND [ARG]...]

COMMAND: nome del processo

senza argomenti: stampa il valore di niceness

-n, --adjustment=N aggiunge l'intero N al valore di niceness

Il comando nice può essere utilizzato da qualsiasi utente, ma solo i processi di proprietà dell'utente stesso possono essere modificati.

Comandi di visualizzazione e analisi dei processi

	<i>Arresto sistema</i>
<i>ps</i>	<i>Visualizzazione processi</i>
<i>ps tree</i>	<i>Visualizzazione processi ad albero</i>
<i>top</i>	<i>Visualizzazione processi e risorse</i>
<i>vmstat</i>	<i>Rapporti sull'occupazione di memoria</i>

ps

Process status - Visualizza una fotografia dei processi presenti sul sistema e alcune proprietà come il PID, lo stato, la memoria occupata, il CPU time attivato ecc.

sintassi: `ps [OPTION]`

- e Visualizza i processi in esecuzione nel sistema
- f Visualizza una lista di processi con informazioni aggiuntive
- u mostra le informazioni relative all'utente proprietario del processo
- x mostra anche i processi che non sono associati a un terminale



EXAMPLES

To see every process on the system using standard syntax:

```
ps -e  
ps -ef  
ps -eF  
ps -ely
```

To see every process on the system using BSD syntax:

```
ps ax  
ps axu
```

To print a process tree:

```
ps -ejH  
ps axjf
```

To get info about threads:

```
ps -eLf  
ps axms
```

To get security info:

```
ps -eo euser,ruser,suser,fuser,f,comm,label  
ps axZ  
ps -eM
```

To see every process running as root (real & effective ID) in user format:

```
ps -U root -u root u
```

To see every process with a user-defined format:

```
ps -eo pid,tid,class,rtprio,ni,pri,psr,pcpu,stat,wchan:14,comm  
ps axo stat,euid,ruid,tt,tpgid,sess,pgrp,ppid,pid,pcpu,comm  
ps -Ao pid,tt,user,fname,tmout,f,wchan
```

Print only the process IDs of syslogd:

```
ps -C syslogd -o pid=
```

Print only the name of PID 42:

```
ps -q 42 -o comm=
```

pstree

Process status tree - Visualizza la gerarchia dei processi del sistema con una struttura ad albero, evidenziando parent e child

sintassi: *ps*tree [OPTION]

- a visualizza le informazioni sui processi come gli argomenti della riga di comando
- c visualizza il nome del programma invece del nome del processo
- p visualizza i PID dei processi
- u visualizza i nomi degli utenti proprietari dei processi
- h Visualizza l'albero in formato JSON

top

Visualizza l'utilizzo delle risorse di CPU in tempo reale, attraverso una tabella, cioè dei processi che la impegnano maggiormente

sintassi: `top [OPTION]`

- d specifica l'intervallo di aggiornamento `top -d 1` #aggiorna ogni secondo
- n specifica il numero di iterazioni da eseguire
- p visualizza i PID dei processi
- u visualizza solo i processi degli utenti specificati
- p visualizza solo i processi specificati

È particolarmente utile per individuare i processi che consumano molte risorse del sistema o che causano problemi di prestazioni.

Le informazioni visualizzate dal comando `top` possono essere aggiornate continuamente o in base a un intervallo di tempo specifico

vmstat (virtual memory statistics)

visualizza in tempo reale informazioni sullo stato del sistema, come la quantità di memoria virtuale utilizzata, le statistiche di CPU, le operazioni di paging di disco e l'utilizzo dei dispositivi di I/O.

sintassi: `vmstat [options] [delay [count]]`

delay tempo di aggiornamento in secondi

count numero di iterazioni

- s mostra informazioni dettagliate sulle attività del sistema
- t aggiunge una colonna che mostra il timestamp dell'output
- d visualizza tutte le statistiche dei dischi di Linux.

E' particolarmente utile durante le attività di debug di sistema o di analisi delle prestazioni del sistema.
E' possibile specificare il tempo di intervallo fra i vari rapporti e il numero massimo degli stessi

vmstat (virtual memory statistics)

```
procs -----memory----- ---swap-- -----io---- -system-- -----cpu-----  
r  b  swpd  free  buff  cache  si  so  bi  bo  in  cs  us  sy  id  wa  st
```

procs: il numero di processi in esecuzione

r: il numero di processi in attesa di esecuzione

b: il numero di processi bloccati in attesa di I/O

memory: la quantità di memoria utilizzata dal sistema

swpd: la quantità di memoria del disco virtuale utilizzata

free: la quantità di memoria libera nel sistema

buff: la quantità di memoria utilizzata per la cache dei buffer del disco

cache: la quantità di memoria utilizzata per la cache del filesystem.

swap: la quantità di spazio di swap utilizzato dal sistema

io: il numero di operazioni di input/output in corso

system: il tempo di sistema utilizzato per eseguire operazioni di sistema

in: numero delle interruzioni per secondo

cs: numero di cambiamenti di contesto per secondo

cpu: l'utilizzo della CPU diviso tra CPU idle, CPU user e CPU kernel

us: percentuale della CPU assegnato ad utente (user mode)

sy: percentuale della CPU assegnato al sistema (kernel mode)

id: percentuale della CPU in idle

wa: percentuale della CPU in attesa degli I/O

st: percentuale della CPU in attesa degli I/O in stato «interruptable sleep»

Codici di stato dei processi linux

Sono valori numerici che indicano lo stato di un processo dopo che è stato terminato o interrotto in modo anomalo.

Vengono utilizzati per determinare se un processo è stato completato correttamente o se si è verificato un errore.

Può essere consultato utilizzando il comando "**echo \$?**"

Codici di stato dei processi linux

0 - il processo è terminato correttamente
1 - errore generico nel processo
2 - errore di sintassi nei parametri del processo
3 - processo interrotto da segnale SIGHUP
4 - processo interrotto da segnale SIGINT
5 - processo interrotto da segnale SIGQUIT
6 - processo interrotto da segnale SIGABRT
7 - processo interrotto da segnale SIGPIPE
8 - errore di allocazione di memoria nel processo
9 - processo interrotto da segnale SIGKILL
10 - processo interrotto da segnale SIGUSR1
11 - processo interrotto da segnale SIGSEGV
12 - processo interrotto da segnale SIGUSR2
13 - processo interrotto da segnale SIGTERM
14 - processo interrotto da segnale SIGCHLD
15 - processo interrotto da segnale SIGCONT
16 - processo interrotto da segnale SIGSTOP

17 - processo interrotto da segnale SIGTSTP
18 - processo interrotto da segnale SIGTTIN
19 - processo interrotto da segnale SIGTTOU
20 - processo interrotto da segnale SIGURG
21 - processo interrotto da segnale SIGPOLL
22 - processo interrotto da segnale SIGXCPU
23 - processo interrotto da segnale SIGXFSZ
24 - processo interrotto da segnale SIGVTALRM
25 - processo interrotto da segnale SIGPROF
26 - processo interrotto da segnale SIGWINCH
27 - processo interrotto da segnale SIGIO
28 - processo interrotto da segnale SIGPWR
29 - processo interrotto da segnale SIGSYS
30 - processo interrotto da segnale SIGRTMIN
31 - processo interrotto da segnale SIGRTMAX
127: il comando in esecuzione non esiste o non è stato trovato
130: il processo è stato interrotto da un segnale SIGINT
(utilizzato spesso con il comando Ctrl-C).

descrizione

rappresentano tutti i processi generati da un unico comando tramite la shell stessa.
Può essere un'applicazione, uno script o comunque una serie di comandi che vengono eseguiti in sequenza.

L'attività di un job può avvenire in:

foregorund (primo piano) – *fg*

background (in sottofondo) – *bg*

Sintassi job in background: **<nome_comando> &** ("&" alla fine del comando)

Se in background, cioè senza esibire la loro uscita direttamente a video, permettono allo user di continuare ad interagire con il sistema in modo interattivo mentre il lavoro procede in background.

Jobs Elenca i job in esecuzione in background, fornendo il rispettivo numero.

sintassi: *jobs -x command [option]*

- l Elenca gli ID di processo oltre le normali informazioni
- p elenca solo gli ID di processo

fg modifica l'esecuzione di un job da background (sfondo) in foreground (primo piano).

sintassi: *fg [job_spec]*

bg fa ripartire un job che era stato sospeso, mettendolo in esecuzione in background.

sintassi: *fg [job_spec]*

disown Cancella il/i job dalla tabella dei job attivi della shell.

Sintassi: *disown [-h] [-ar] [jobspec ... | pid ...]*

wait Arresta l'esecuzione dello script finché tutti i job in esecuzione in background non sono terminati, o finché non è terminato il job o il processo il cui ID è stato passato come opzione.

può essere usato per evitare che uno script termini prima che un job in esecuzione in background abbia ultimato il suo compito

Comunicazione tra processi

Nel momento in cui l'attività di un processo dipende da quella di un altro ci deve essere una forma di comunicazione tra i due

INVIO SEGNALI

usati anche dal kernel per comunicare ai processi una serie di eventi o errori

SEGNALI CONCLUSIONE

Segnale	Descrizione
SIGINT	È un segnale di interruzione intercettabile, inviato normalmente attraverso la tastiera del terminale, con la combinazione [<i>Ctrl c</i>], al processo che si trova a funzionare in primo piano (<i>foreground</i>). Di solito, il processo che riceve questo segnale viene interrotto.
SIGQUIT	È un segnale di interruzione intercettabile, inviato normalmente attraverso la tastiera del terminale, con la combinazione [<i>Ctrl \</i>], al processo che si trova a funzionare in primo piano. Di solito, il processo che riceve questo segnale viene interrotto.
SIGTERM	È un segnale di conclusione intercettabile, inviato normalmente da un altro processo. Di solito, provoca la conclusione del processo che ne è il destinatario.
SIGKILL	È un segnale di interruzione non intercettabile, che provoca la conclusione immediata del processo. Non c'è modo per il processo destinatario di eseguire alcuna operazione di salvataggio o di scarico dei dati.

Segnali gestiti da linux

- **A** termina il processo;
- **B** il segnale viene ignorato;
- **C** la memoria viene scaricata (*core dump*);
- **D** il processo viene fermato;
- **E** il segnale non può essere catturato;
- **F** il segnale non può essere ignorato.

Segnale	Azione	Descrizione
SIGHUP	A	Il collegamento con il terminale è stato interrotto.
SIGINT	A	Interruzione attraverso un comando dalla tastiera.
SIGQUIT	A	Conclusione attraverso un comando dalla tastiera.
SIGILL	A	Istruzione non valida.
SIGABRT	C	Interruzioni di sistema.
SIGFPE	C	Eccezione in virgola mobile.
SIGKILL	AEF	Conclusione immediata.
SIGSEGV	C	Riferimento non valido a un segmento di memoria.
SIGPIPE	A	<i>Pipe</i> interrotta.
SIGALRM	A	Timer.
SIGTERM	A	Conclusione.
SIGUSR1	A	Primo segnale definibile dall'utente.
SIGUSR2	A	Secondo segnale definibile dall'utente.
SIGCHLD	B	Eliminazione di un processo figlio.
SIGCONT		Riprende l'esecuzione se era stato fermato.
SIGTSTOP	DEF	Ferma immediatamente il processo.
SIGTSTP	D	Stop attraverso un comando della tastiera.
SIGTTIN	D	Processo in <i>background</i> che necessita dell'input.
SIGTTOU	D	Processo in <i>background</i> che necessita dell'output.

Comunicazione tra processi

Si possono inviare segnali ai processi: attraverso vari comandi di shell, oppure da tastiera

COMANDO

kill è un comando che permette di inviare segnali ai processi:

kill [opzioni] PID

Per avere un elenco dei segnali: ***kill -l***

Per inviare un segnale: ***kill -s segnale PID***

TASTIERA

Per inviare segnali da tastiera:

inviare un SIGINT: **ctrl+c**

inviare un SIGQUIT: **ctrl+**

inviare un SIGTSTP: **ctrl+z**

inviare un SIGCONT: **fg**

KILL

permette di inviare segnali ai processi

sintassi: *kill [OPTION] PID*

PID Specifica la lista di processi ai quali kill deve inviare il segnale.

opzioni:

- n** il processo con pid n viene killato (n maggiore di 0)
- 0** tutti i processi nel gruppo processi corrente vengono killati
- 1** tutti i processi con pid maggiore di 1 vengono killati
- n** tutti i processi nel gruppo processi n vengono killati (n maggiore di 0)
- name** tutti i processi che hanno nome «name» vengono killati

Kill -I

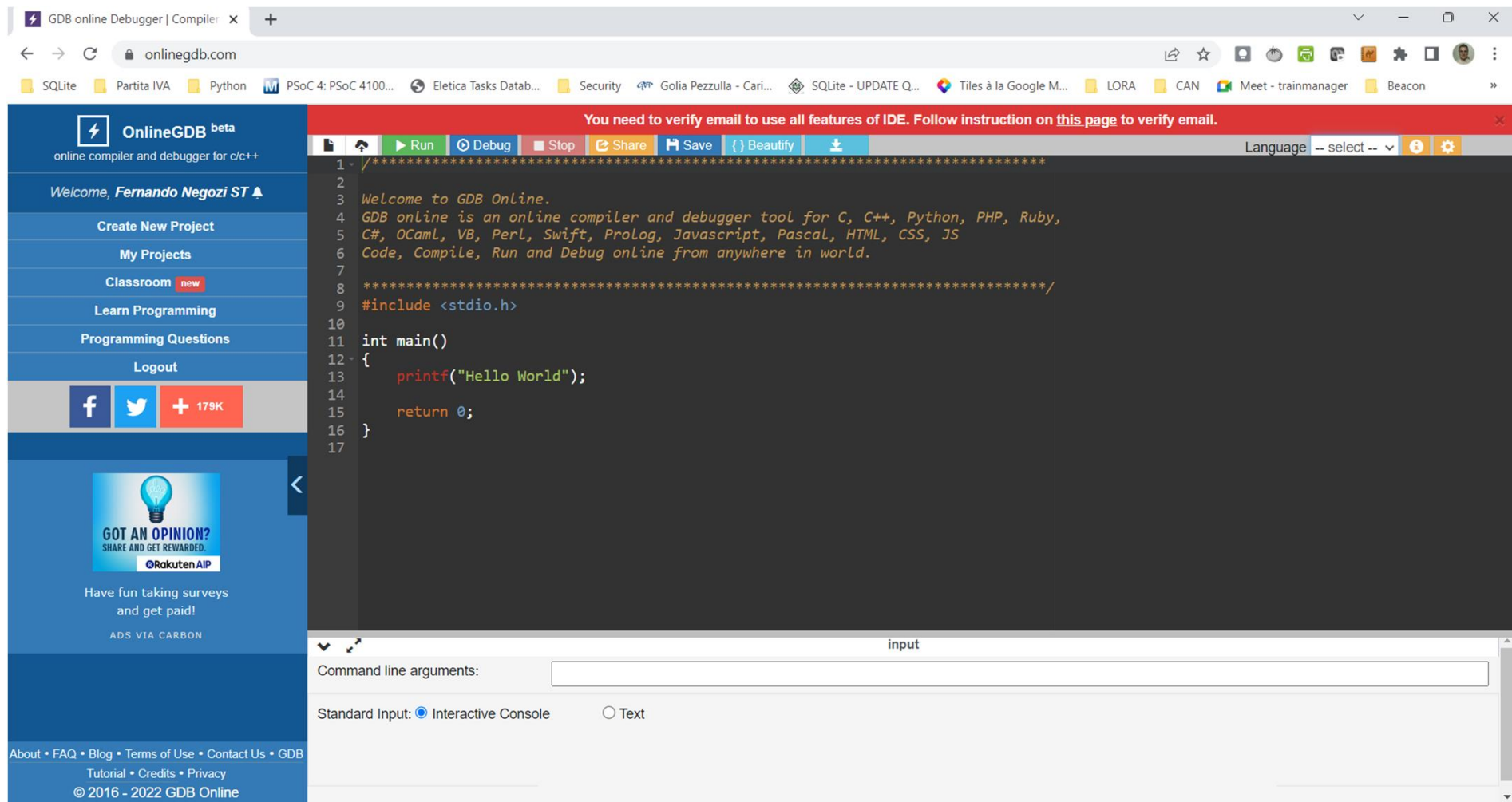
Elenco dei segnali

1) SIGHUP	2) SIGINT	3) SIGQUIT	4) SIGILL	5) SIGTRAP
6) SIGABRT	7) SIGBUS	8) SIGFPE	9) SIGKILL	10) SIGUSR1
11) SIGSEGV	12) SIGUSR2	13) SIGPIPE	14) SIGALRM	15) SIGTERM
16) SIGSTKFLT	17) SIGCHLD	18) SIGCONT	19) SIGSTOP	20) SIGTSTP
21) SIGTTIN	22) SIGTTOU	23) SIGURG	24) SIGXCPU	25) SIGXFSZ
26) SIGVTALRM	27) SIGPROF	28) SIGWINCH	29) SIGIO	30) SIGPWR
31) SIGSYS	34) SIGRTMIN	35) SIGRTMIN+1	36) SIGRTMIN+2	37) SIGRTMIN+3
38) SIGRTMIN+4	39) SIGRTMIN+5	40) SIGRTMIN+6	41) SIGRTMIN+7	42) SIGRTMIN+8
43) SIGRTMIN+9	44) SIGRTMIN+10	45) SIGRTMIN+11	46) SIGRTMIN+12	47) SIGRTMIN+13
48) SIGRTMIN+14	49) SIGRTMIN+15	50) SIGRTMAX-14	51) SIGRTMAX-13	52) SIGRTMAX-12
53) SIGRTMAX-11	54) SIGRTMAX-10	55) SIGRTMAX-9	56) SIGRTMAX-8	57) SIGRTMAX-7
58) SIGRTMAX-6	59) SIGRTMAX-5	60) SIGRTMAX-4	61) SIGRTMAX-3	62) SIGRTMAX-2
63) SIGRTMAX-1	64) SIGRTMAX			



gdbonline

<https://www.onlinegdb.com>



The screenshot shows the OnlineGDB website interface. The browser address bar displays "onlinegdb.com". The page features a sidebar on the left with navigation links: "Create New Project", "My Projects", "Classroom", "Learn Programming", "Programming Questions", and "Logout". Below these are social media icons for Facebook and Twitter, and a "179K" follower count. The main content area has a red banner at the top stating "You need to verify email to use all features of IDE. Follow instruction on this page to verify email." Below the banner is a code editor with a dark background and syntax-highlighted C code. The code includes a welcome message and a "Hello World" program. The bottom of the interface shows a command line input field and a radio button selection for "Standard Input" (Interactive Console or Text).

```
1- /*****  
2-  
3- Welcome to GDB Online.  
4- GDB online is an online compiler and debugger tool for C, C++, Python, PHP, Ruby,  
5- C#, OCaml, VB, Perl, Swift, Prolog, Javascript, Pascal, HTML, CSS, JS  
6- Code, Compile, Run and Debug online from anywhere in world.  
7-  
8- *****/  
9- #include <stdio.h>  
10-  
11- int main()  
12- {  
13-     printf("Hello World");  
14-  
15-     return 0;  
16- }  
17-
```



 **OnlineGDB** beta
online compiler and debugger for c/c++

Welcome, **Fernando Negozi ST** ▲

- Create New Project
- My Projects
- Classroom** new
- Learn Programming
- Programming Questions
- Logout

Early access of onlinedb classroom

Yay! You have been given early access of **onlinedb classroom**.

With onlinedb classroom,
Teacher can publish programming assignment amongst the students and can receive submission for evaluation.

To begin with, fill up below information:

Are you student or teacher?

☒ Student ← 2

☐ Teacher

School/University/Institute Name:

Select School/University/Institute ← 3 Add New

Submit

School/University/Institute Name:

Università Politecnica delle Marche, Ancor ▲



Configurazione
interfaccia

BARRA COMANDI

Selezione
linguaggio

Pagina scrittura codice

GDB online Debugger | Compile: x

onlinegdb.com

SQLite Partita IVA Python PSoc 4: PSoc 4100...

OnlineGDB beta
online compiler and debugger for c/c++

Welcome, **Fernando Negozi ST**

Create New Project

My Projects

Classroom **new**

Learn Programming

Programming Questions

Logout

f t + 179K

GOT AN OPINION?
SHARE AND GET REWARDED.
RakutenAIP

Have fun taking surveys
and get paid!
ADS VIA CARBON

About • FAQ • Blog • Terms of Use • Contact Us • GDB
Tutorial • Credits • Privacy
© 2016 - 2022 GDB Online

1 *****

2

3 Welcome to GDB Online.

4 GDB online is an online compiler and debugger tool for C, C++, Python, PHP, Ruby,

5 C#, OCaml, VB, Perl, Swift, Prolog, Javascript, Pascal, HTML, CSS, JS

6 Code, Compile, Run and Debug online from anywhere in world.

7

8 *****/

9 #include <stdio.h>

10

11 int main()

12 {

13 printf("Hello World");

14

15 return 0;

16 }

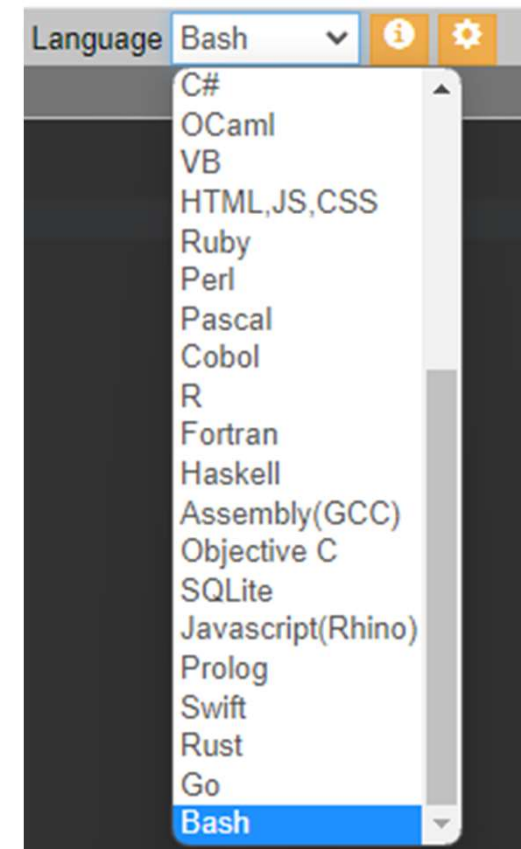
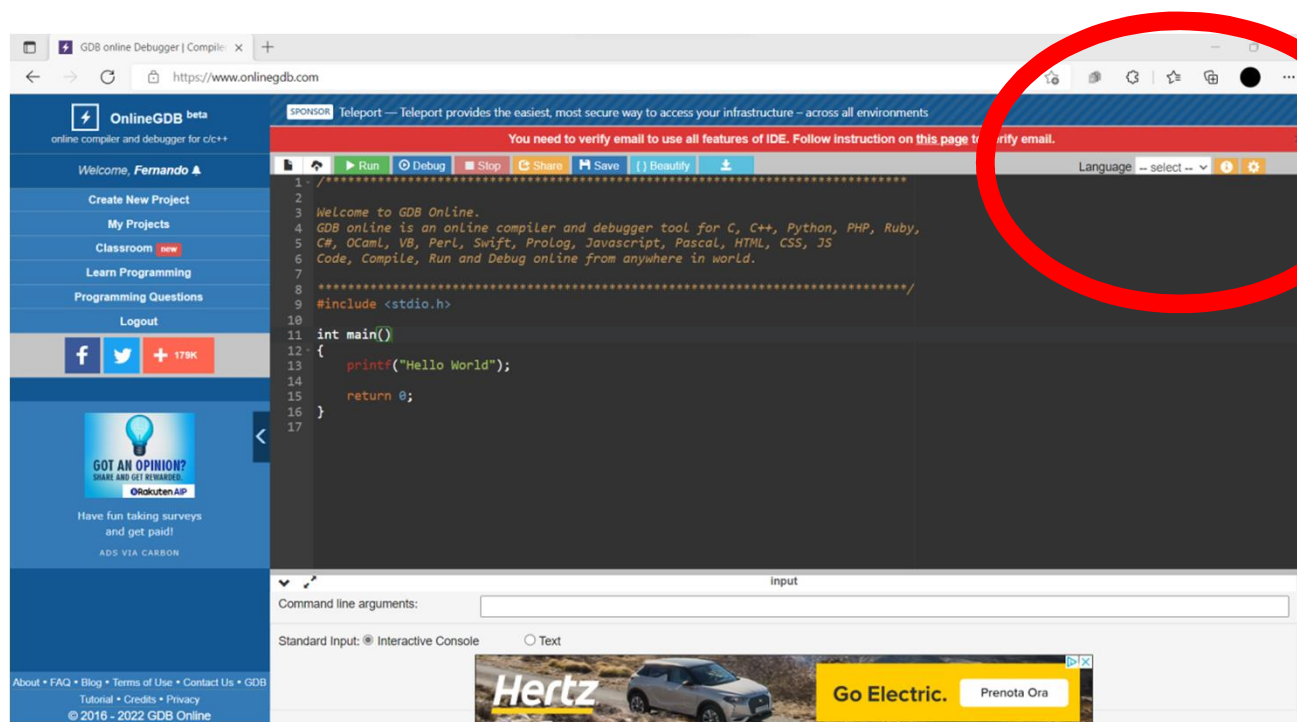
17

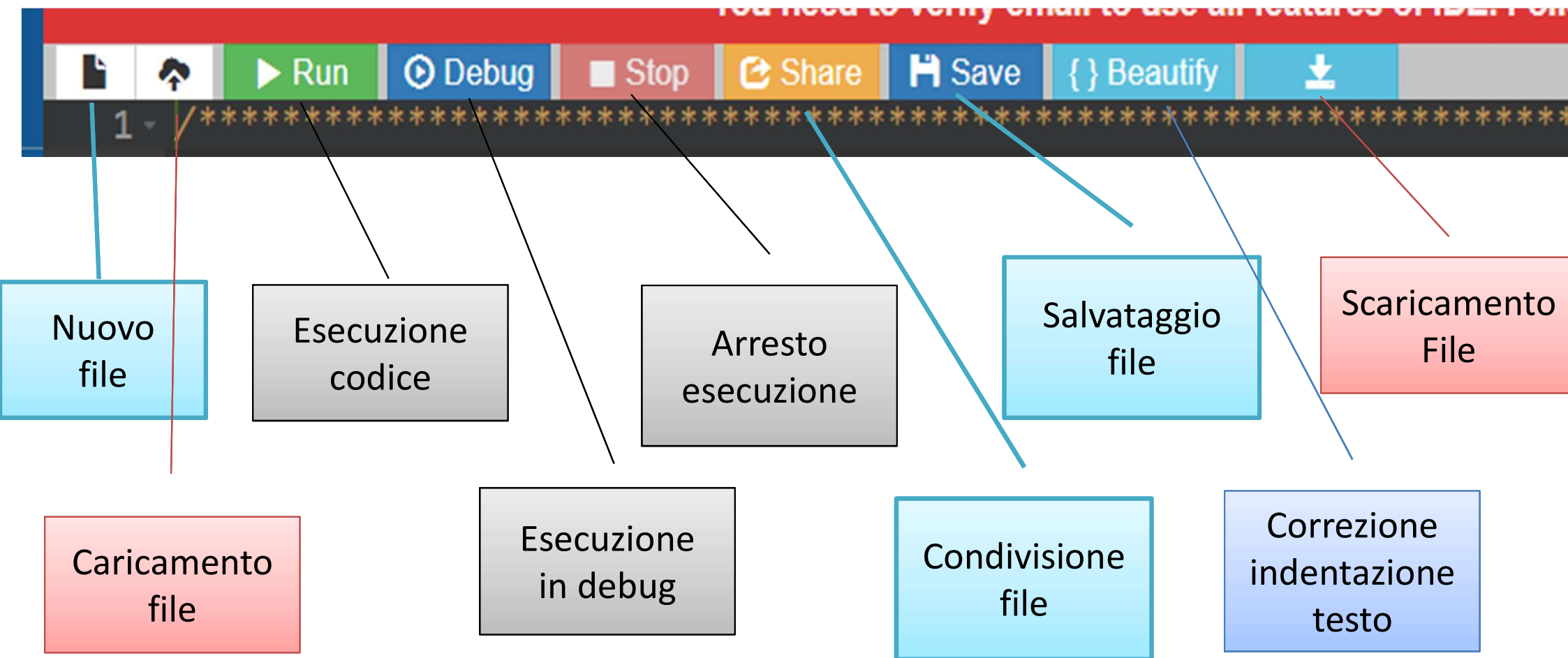
Language --select--

input

Command line arguments:

Standard Input: ☒ Interactive Console ☐ Text







```
main.bash
1 #
2 # Welcome to GDB Online.
3 # GDB online is an online compiler and debugger tool for C, C++, Python, Java, PHP, Ruby, Perl,
4 # C#, OCaml, VB, Swift, Pascal, Fortran, Haskell, Objective-C, Assembly, HTML, CSS, JS, SQLite, Prolog.
5 # Code, Compile, Run and Debug online from anywhere in world.
6 #
7 #
8 echo "Hello World";

Hello World

...Program finished with exit code 0
Press ENTER to exit console.
```

Finestra di interfaccia



Esercizi

#!/bin/bash

#esercizio5_1 - Creazione file di testo

#Creare una directory chiamata "test":

#Entrare nella directory appena creata:

#Creare un file vuoto chiamato "file.txt":

#Scrivere "Ciao Mondo" nel file appena creato:

#Visualizzare il contenuto del file:

#Rinominare il file "file.txt" in "nuovo_file.txt":

#Copiare il file nella directory superiore:

#spostarsi nella cartella superiore:

#Eliminare la directory "test" e il suo contenuto:

#!/bin/bash

#esercizio_5_2 - Creazione file di testo

#partendo da una cartella esercizi

#Visualizzare il contenuto della cartella:

#creare una nuova cartella eser_5_2°

#creare una nuova cartella eser_5_2B

#spostarsi nella cartella lavoro eser_5_2A:

#creare un file "filemove.txt" in questa cartella

#copiare il file nella cartella eser_5_2B:

#Rinominare il file nella cartella eser_5_2B in filemoveB.txt: